

Perceptrón multicapa y back-propagation

Inteligencia Computacional · FICH-UNL · Diego Milone
Diapositivas 30–90 de *Perceptrón multicapa* y transcripciones de clase 009 a 014

Índice

0. Cómo usar este apunte	2
1. Qué resuelve cada arquitectura	2
2. Arquitectura general y notación	6
3. Propagación hacia adelante	7
4. La función de activación: sigmoide simétrica	8
5. El criterio de error	10
6. La regla del gradiente y la cadena que hay que derivar	10
7. El factor fácil: $\partial v_j / \partial w_{ji} = y_i$	12
8. El gradiente de error local instantáneo: δ_j	13
9. La derivada de la sigmoide simétrica	14
10. La regla de actualización	17
11. Back-propagation en la capa de salida	18
12. Back-propagation en las capas ocultas	20
13. La generalización: una capa p cualquiera	24
14. El algoritmo completo, paso a paso	25
15. Cierre: el recorrido completo de la unidad	29
Formulario	30
Errores típicos	31
Autoevaluación	32

0. Cómo usar este apunte

Continúa a **03-xor-con-tres-neuronas.md**, donde se armó a mano una red de tres neuronas que resuelve el XOR. Ahí quedaron dos preguntas abiertas, y son las que esta parte responde:

1. ¿Qué se puede resolver en general, según cuántas capas tenga la red?
2. ¿Cómo hace la red para encontrar sola los pesos que allá pusimos a mano?

Mismo formato que los apuntes anteriores: al final de cada sección hay una **tabla de claves** para taparse el cuerpo y responder en voz alta, y los recuadros **IDEA DE FONDO**, **OJO** y **PARA LA DEFENSA** son los detalles finos.

Notación: vectores columna, $x_0 = -1$ como entrada de sesgo, $\langle w, x \rangle = w^T x$. A partir de acá se agregan **negritas para matrices y vectores** (W, y) y **superíndices romanos para la capa** (W^I, y^{II}).

1. Qué resuelve cada arquitectura

Éste es un resultado importante de los años 80 y la figura está en toda la bibliografía. Resume qué tipo de regiones de decisión puede construir una red según cuántas capas tenga.

Una capa — semiplano

Un solo perceptrón traza una recta y parte el espacio en dos: $+1$ de un lado, -1 del otro. En general, con N entradas, un **hiperplano en $\mathbb{R}^N$** .

Falla en el XOR, y falla mucho peor con clases entrelazadas: por más que se acomode la recta, siempre queda una porción de una clase del lado equivocado.

Dos capas — regiones convexas

Es la red del apunte anterior. Cada neurona de la primera capa aporta un semiplano; la de salida los **interseca**. Con más neuronas en la primera capa se intersecan más semiplanos y la región se afina — de una franja se pasa a un triángulo, a un pentágono, a algo cada vez más redondeado.

Pero hay un techo: **las regiones son siempre convexas** (abiertas o cerradas). No se pueden hacer regiones con huecos ni con concavidades, y por eso las dos medialunas siguen sin resolverse: la parte cóncava de una clase queda inevitablemente adentro de la región de la otra.

Tres capas — regiones arbitrarias

Al agregar una capa más aparece una **capa oculta**, que se llama así porque “no está en contacto ni con la salida ni con la entrada”. Con eso se pueden construir regiones con **concavidades**, con **huecos**, e incluso regiones **no conexas** (dos manchas separadas que pertenecen a la misma clase). Ahí sí entran el hueco entre las dos medialunas y las dos esquinas sueltas del XOR.

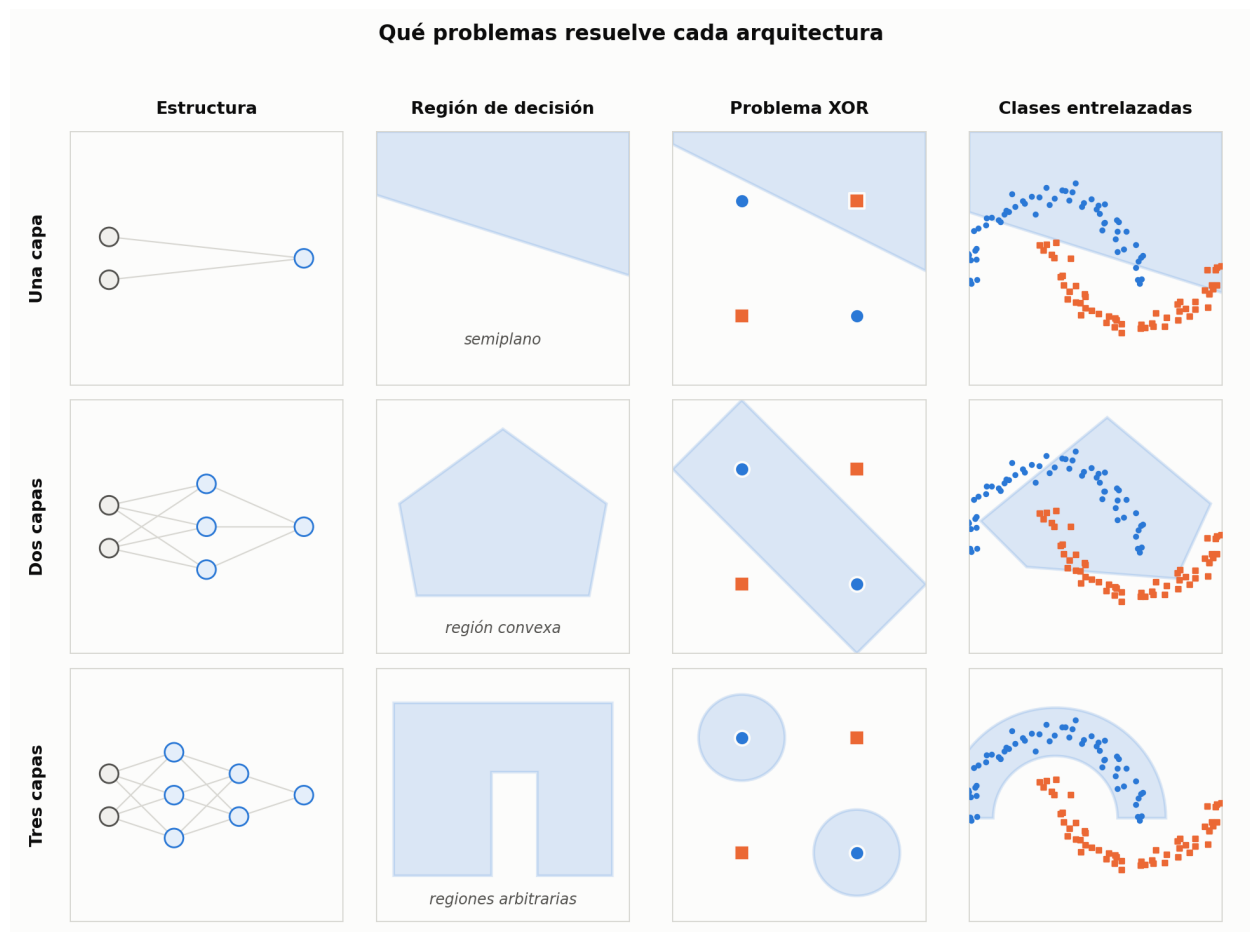


Figura 1: Cada fila agrega una capa y con ella un tipo de región nueva. Las columnas 3 y 4 muestran dos problemas concretos: el XOR y dos clases entrelazadas.

OJO — cómo cuenta las capas la cátedra

Acá “capas” son **capas de neuronas**, no de valores. Las entradas $x_1, x_2, \dots$ **no cuentan como capa**: son datos, no neuronas. Por eso la red del XOR —dos neuronas más una de salida— es de **dos capas**, no de tres. Mucha bibliografía moderna la llamaría “una capa oculta” o incluso “de tres capas” contando la entrada. **No es un error de nadie: son convenciones distintas**. Fijate siempre cómo cuenta el texto que estás leyendo antes de comparar resultados.

PARA LA DEFENSA — “virtualmente cualquier problema” tiene letra chica

La frase que se usa en clase es que tres capas permiten resolver *virtualmente cualquier problema*, y enseguida viene la aclaración: “**la arquitectura permite resolverlo, pero hay que saber encontrar los pesos que efectivamente lo resuelven**”. Son dos cosas distintas y conviene decirlas separadas: **existencia** (¿hay una configuración de pesos que resuelve esto?) y **aprendizaje** (¿el algoritmo la encuentra?). El resultado de los años 80 es sobre lo primero. Todo lo que viene después —back-propagation, la elección de μ , los mínimos locales— es el problema de lo segundo, y no está garantizado.

Cómo elegir la arquitectura para un problema concreto

La tabla dice qué puede hacer cada arquitectura, pero en un parcial o en un TP la pregunta viene al revés: *dado este problema, ¿qué red hace falta?* El razonamiento tiene tres pasos.

Paso 1 — ¿qué forma tiene la región que necesito? Mirá dónde está cada clase y preguntate si la frontera que las separa es una recta, un contorno cerrado, o algo con entrantes.

Paso 2 — buscá esa forma en la tabla. Recta abierta → una capa. Región convexa, abierta o cerrada → dos capas. Con concavidades, huecos o partes separadas → tres.

Paso 3 — cuántas neuronas en la capa oculta. Cada neurona aporta un semiplano y la capa siguiente los interseca, así que N neuronas ocultas dan un polígono de hasta N lados. De ahí salen dos consecuencias: con $N = 2$ **no se puede cerrar** una región (dos semiplanos dan una franja o una cuña, nunca un recinto), y a más neuronas, más lados y mejor aproximación a un contorno curvo.

Los tres casos clásicos, que son los de la figura:

Problema	Región que hace falta	Arquitectura mínima	Por qué
XOR	una franja entre dos rectas paralelas	2 capas, 2 ocultas	convexa abierta : alcanza con intersectar dos semiplanos
Clases concéntricas	un recinto cerrado alrededor de la clase interior	2 capas, 3 ocultas o más	convexa cerrada : hacen falta al menos 3 semiplanos para cerrar un polígono

Medialunas	un contorno con una concavidad	3 capas	ninguna región convexa puede seguir la curva de una medialuna sin comerse parte de la otra
-------------------	---------------------------------------	---------	--

IDEA DE FONDO — el XOR y las concéntricas son el mismo caso, del derecho y del revés

Los dos se resuelven con **una sola capa oculta**, y sin embargo parecen problemas muy distintos. La diferencia es sólo si la región convexa queda **abierta** (la franja del XOR) o **cerrada** (el recinto de las concéntricas). Ésa es toda la variedad que da una capa oculta, y es más de lo que parece. Lo que **no** puede hacer, por más neuronas que le pongas, es una concavidad — y ahí es donde recién hace falta la tercera capa.

OJO — el mínimo geométrico no es el mínimo práctico

El paso 3 te da el **mínimo teórico**, o sea cuántas neuronas hacen falta para que la solución *exista*. Que el entrenamiento la *encuentre* es otra cosa: con el mínimo justo, la convergencia depende mucho de la inicialización. La regla de trabajo es ponerle unas cuantas neuronas más que el mínimo. No cambia lo que la red **puede** representar — cambia la probabilidad de que back-propagation llegue.

Claves de la sección 1

Clave	Qué tenés que poder responder
Una capa	Qué región genera y por qué falla en el XOR
Dos capas	Qué operación hace la capa de salida sobre los semiplanos
Límite convexo	Qué tipo de región no se puede hacer con dos capas
Tres capas	Qué habilita la capa oculta que antes no se podía
Elegir arquitectura	Los tres pasos, y aplicarlos al XOR, a las concéntricas y a las medialunas
Cuántas neuronas	Por qué N ocultas dan un polígono de N lados y por qué con 2 no se cierra
Conteo de capas	Por qué la red del XOR es “de dos capas” en esta materia
Letra chica	Diferencia entre que exista solución y que el algoritmo la encuentre

2. Arquitectura general y notación

Hasta acá las redes eran de juguete. Ahora se generaliza a cualquier cantidad de entradas, neuronas y capas.

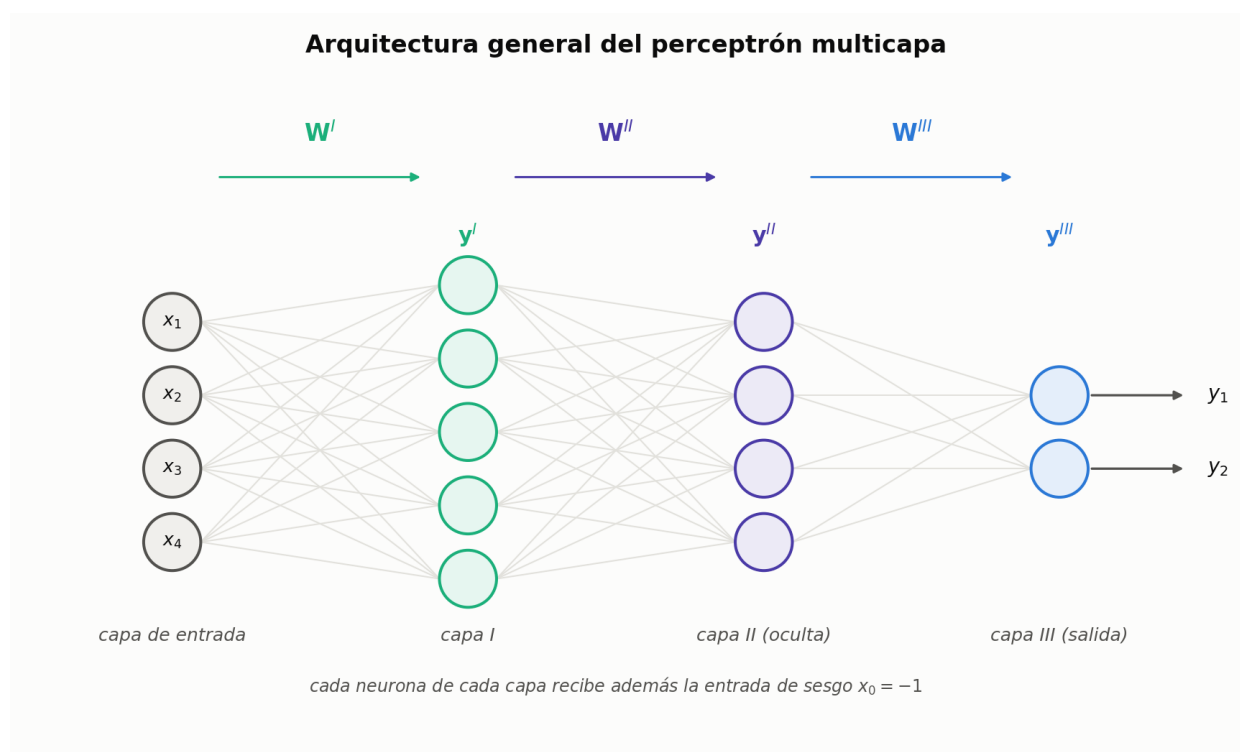


Figura 2: Tres capas de neuronas, una matriz de pesos por capa y un vector de salidas por capa.

Tres cosas cambian respecto de los ejemplos anteriores:

Muchas entradas. El dibujo usa cuatro, o sea patrones en $\mathbb{R}^4$, pero no hay límite. El ejemplo que se da en clase es contundente: si la entrada es una imagen de 1024×1024 y cada píxel es una entrada, la red está clasificando en $\mathbb{R}^{1\,000\,000}$.

Muchas salidas. Con una salida ± 1 había dos clases y nada más. Con dos salidas hay cuatro combinaciones, con tres hay ocho: **ya no se está limitado a dos clases.**

Notación matricial. Los pesos de cada capa se agrupan en una matriz y las salidas en un vector:

$$\mathbf{W}^I, \mathbf{y}^I \quad \mathbf{W}^{II}, \mathbf{y}^{II} \quad \mathbf{W}^{III}, \mathbf{y}^{III}$$

OJO — el superíndice es de la capa de llegada

$\mathbf{W}^{II}$ no son los pesos “de la segunda capa de conexiones vista desde la entrada”: son los pesos que **unen la capa I con la capa II**, y se le adjudican a la capa II. La regla es: *la notación hace referencia siempre a la capa de la neurona en la que estamos parados*. Los pesos que entran a una neurona pertenecen a la capa de esa neurona.

Dimensiones. Si la capa p tiene M_p neuronas y recibe M_{p-1} entradas, entonces

$$\mathbf{W}^{(p)} \text{ es de } M_p \times (M_{p-1} + 1)$$

El +1 es la columna del sesgo. En el dibujo, con 4 entradas y 5 neuronas en la capa I, se cuentan **20 conexiones dibujadas** — pero la matriz real es de $5 \times 5 = 25$ pesos, porque el dibujo omite el x_0 para no ensuciarlo.

OJO — el sesgo está en todas las capas

En los diagramas generales casi nunca se dibuja, pero **la entrada $x_0 = -1$ está conectada a todas las neuronas de todas las capas**, cada una con su propio w_0 . Que no aparezca en el dibujo no significa que no esté: significa que se lo da por sobreentendido. Al contar parámetros o al programar, es el error más fácil de cometer.

Claves de la sección 2

Clave	Qué tenés que poder responder
$\mathbf{W}^{(p)}$	A qué capa pertenece una matriz de pesos
Dimensiones	Cuántas filas y columnas tiene, y de dónde sale el +1
Muchas salidas	Por qué dejan de ser sólo dos clases
Sesgo	Dónde está aunque no se dibuje
Espacio de entrada	Por qué $\mathbb{R}^{1\,000\,000}$ no es un ejemplo exótico

3. Propagación hacia adelante

Con la notación armada, calcular la salida de la red es repetir tres veces la misma operación.

Capa I. Para la neurona j , producto interno entre sus pesos y la entrada:

$$v_j^I = \langle \mathbf{w}_j^I, \mathbf{x} \rangle = \sum_{i=0}^N w_{ji}^I x_i \quad (\text{completo: } \mathbf{v}^I = \mathbf{W}^I \mathbf{x})$$

A v se le llama **salida lineal**, porque todavía no pasó por la función de activación. Después:

$$y_j^I = \varphi(v_j^I)$$

Capa II. Toma como entrada las salidas de la capa I:

$$v_j^{II} = \langle \mathbf{w}_j^{II}, \mathbf{y}^I \rangle \quad \Rightarrow \quad y_j^{II} = \varphi(v_j^{II})$$

Capa III. Igual, y sus salidas **son** las salidas de la red:

$$v_j^{III} = \langle \mathbf{w}_j^{III}, \mathbf{y}^{II} \rangle \quad \Rightarrow \quad y_j^{III} = \varphi(v_j^{III}) = y_j$$

IDEA DE FONDO — la distinción v / y no es cosmética

Separar la **salida lineal** v de la **salida** y parece un capricho de notación, pero es lo que hace legible toda la derivación de back-propagation. La cadena de derivadas pasa exactamente por ese punto: primero se deriva y respecto de v (ahí aparece φ'), y recién después v respecto de w (ahí aparece la entrada). Si no hubieras separado los dos, no tendrías dónde apoyar la regla de la cadena.

Claves de la sección 3

Clave	Qué tenés que poder responder
v vs. y	Qué es cada una y en qué orden se calculan
Forma matricial	Escribir $\mathbf{v}^I = \mathbf{W}^I \mathbf{x}$ y decir las dimensiones
Encadenado	Qué es la entrada de la capa II y de la capa III
Por qué separarlas	Qué papel juega esa distinción en back-propagation

4. La función de activación: sigmoide simétrica

Acá se abandona la función signo. En su lugar:

$$\varphi(v) = \frac{2}{1 + e^{-bv}} - 1$$

Por qué se cambia. Porque hay que derivarla. Todo el método del gradiente exige que la función de activación sea derivable, y sgn tiene una discontinuidad en el origen: su derivada es cero donde está definida y no existe justo donde importa. La sigmoide da la misma forma en S **sin discontinuidades**.

Qué hace b . Controla cuán abrupta es la transición. Con b grande la curva se pega a la función signo; con b chico es una rampa suave. O sea: *se puede tener el comportamiento del signo sin pagar el precio de la discontinuidad*.

Simétrica. Va de -1 a $+1$, igual que sgn , así que toda la codificación bipolar de las unidades anteriores se mantiene.

OJO — el aviso más importante de esta clase

Existen **dos** sigmoides muy parecidas en la bibliografía:

- la **simétrica**, que es la que usa esta materia, entre -1 y $+1$;
- la **logística**, entre 0 y 1 .

Las dos son correctas —es una cuestión de convención—, pero **si cambiás de una a la otra, todos los cálculos cambian**: la derivada es distinta, las fórmulas de los deltas son distintas, la codificación de las salidas deseadas es distinta. En clase se insiste: *no mezclar ecuaciones de un libro con ecuaciones de otro*, porque después no hay forma de que la red funcione. Antes de copiar una fórmula de Haykin o de Bishop, fijate cuál de las dos usa.

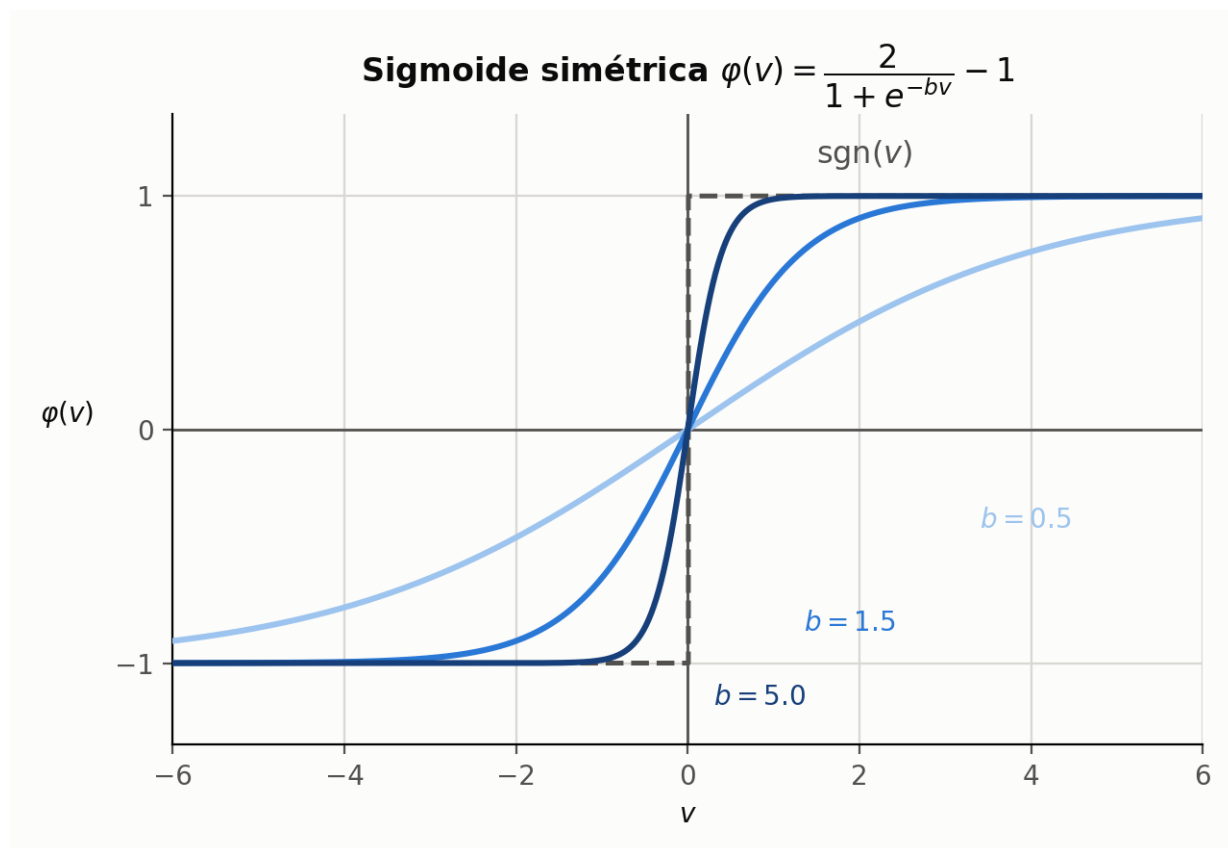


Figura 3: La sigmoide simétrica para tres valores de b , con sgn punteada como referencia.

IDEA DE FONDO — b y la escala de w hacen lo mismo

En el apunte anterior quedó que la escala del vector de pesos era un grado de libertad irrelevante, porque con sgn sólo importa el signo. Con la sigmoide **deja de ser irrelevante**: como φ actúa sobre $v = \langle w, x \rangle$, multiplicar w por k es exactamente lo mismo que multiplicar b por k . Los dos controlan la misma cosa: cuán abrupta es la decisión. Consecuencia práctica: normalmente se fija b y se deja que el entrenamiento ajuste $\|w\|$. Y también explica por qué la inicialización de pesos conviene chica: con $\|w\|$ grande la red arranca saturada, en la zona plana de la sigmoide, donde $\varphi' \approx 0$ y por lo tanto **no aprende**.

Claves de la sección 4

Clave	Qué tenés que poder responder
Fórmula	Escribir $\varphi(v)$ de memoria
Motivo del cambio	Por qué sgn no sirve para el gradiente
Parámetro b	Qué controla y qué pasa en los extremos
Las dos sigmoides	En qué se diferencian y por qué no se pueden mezclar
Saturación	Por qué conviene inicializar los pesos chicos

5. El criterio de error

Ahora la red tiene varias salidas, así que hay **un error por cada neurona de salida**. Para cada una:

$$e_j(n) = d_j(n) - y_j(n)$$

y el criterio de error de la red completa es la **suma del error cuadrático instantáneo**:

$$\xi(n) = \frac{1}{2} \sum_{j=1}^M e_j^2(n)$$

Tres decisiones metidas en esa fórmula, y las tres tienen motivo:

Por qué al cuadrado. No es cosmético. Si una salida se equivoca en $+1$ y otra en -1 , al sumarlas **se compensarían** y el error total daría cero, con la red equivocándose en las dos. Elevando al cuadrado, cada error suma siempre y el total sólo baja cuando las neuronas realmente aciertan.

Por qué el $\frac{1}{2}$. Pura conveniencia: cuando se derive, el exponente 2 va a bajar multiplicando y se va a cancelar con este $\frac{1}{2}$. Es un truco de limpieza, no tiene contenido.

Por qué “instantáneo”. Es el error del patrón n , **no** el error sobre todo el conjunto de entrenamiento. El mismo ejemplo puede volver a mostrarse muchas iteraciones después, pero cada vez se calcula su propio $\xi(n)$.

IDEA DE FONDO — dónde se fue el 2μ del perceptrón simple

En la unidad anterior el criterio era $e^2(n)$, sin el medio, y la regla terminaba en $w(n+1) = w(n) + 2\mu e(n) x(n)$ — con ese 2 colgado adelante que después había que absorber en la constante. Con $\xi = \frac{1}{2} \sum e_j^2$, ese 2 se cancela solo y las fórmulas de back-propagation quedan sin factores sueltos. **Es la misma cuenta, con la constante elegida de entrada para que salga limpia.**

Claves de la sección 5

Clave	Qué tenés que poder responder
$\xi(n)$	Escribirla y decir sobre qué índice suma
El cuadrado	El argumento de la compensación entre errores
El $\frac{1}{2}$	Para qué está y de dónde viene
“Instantáneo”	Qué distingue este error del error sobre todo el conjunto

6. La regla del gradiente y la cadena que hay que derivar

La idea es la misma de la unidad anterior: mover cada peso en el sentido opuesto al gradiente del error.

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

Se deriva **respecto de los pesos** porque la superficie de error tiene a los pesos como coordenadas: distintos pesos, distinto error, y lo que se busca es el conjunto de pesos que lo hace mínimo.

Sobre μ , el compromiso se explica bien en clase:

- μ **muy grande**: la red se aprende el ejemplo que le acabás de mostrar, pero “se olvida todo lo que aprendió antes”, porque modifica los pesos tanto que borra el ajuste de los ejemplos anteriores.
- μ **muy chico**: aprende bien pero lentísimo; hay que mostrarle todo muchísimas veces.

La regla de la cadena

Acá está el corazón del asunto. El error no depende de los pesos de forma directa: depende **a través de una cadena de dependencias**.

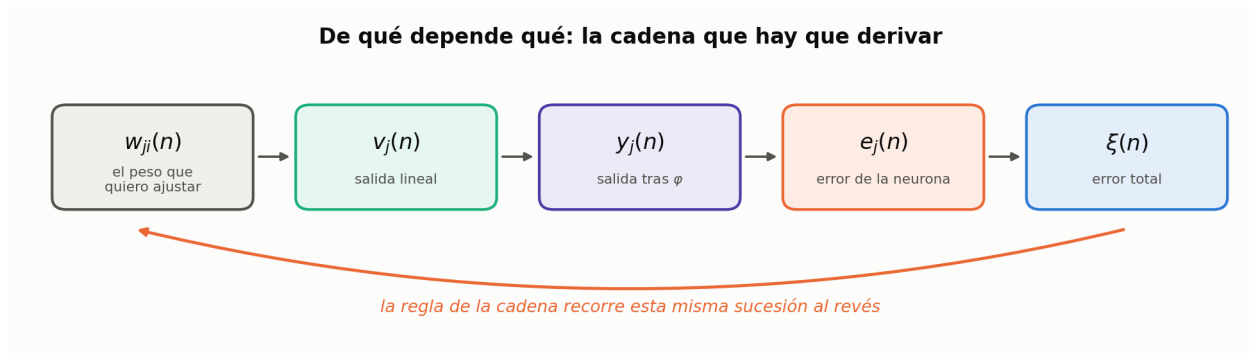


Figura 4: Cada eslabón depende del anterior; derivar ξ respecto de w_{ji} obliga a recorrerlos todos.

Leído de derecha a izquierda, que es como lo dice la clase — “una función de una función de una función”:

- $\xi(n)$ depende de $e_j(n)$, el error de la neurona j ;
- $e_j(n)$ depende de $y_j(n)$, la salida de esa neurona (es una simple resta, pero depende);
- $y_j(n)$ depende de $v_j(n)$, la salida lineal, a través de φ ;
- $v_j(n)$ es lo único que depende **de forma directa** del peso $w_{ji}(n)$.

Por lo tanto:

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

PARA LA DEFENSA — ésta es la ecuación fundamental

Todo lo que sigue en la unidad es **resolver estos cuatro factores uno por uno** y después repetir el procedimiento hacia atrás capa por capa. Si podés escribir esta ecuación y explicar de dónde sale cada factor, tenés el esqueleto completo de back-propagation aunque todavía no hayas hecho las cuentas. Y notá la simetría con la propagación hacia adelante: **la información va $w \rightarrow v \rightarrow y \rightarrow e \rightarrow \xi$, y la**

derivada recorre exactamente el mismo camino al revés. De ahí el nombre: propagación hacia atrás.

Claves de la sección 6

Clave	Qué tenés que poder responder
Regla del gradiente	Escribir Δw_{ji} y justificar el signo
μ grande / chico	Qué falla en cada extremo
La cadena	Nombrar los cuatro eslabones en orden
Ecuación fundamental	Escribirla completa y explicar cada factor
El nombre	Por qué se llama “propagación hacia atrás”

7. El factor fácil: $\partial v_j / \partial w_{ji} = y_i$

Se empieza por el último eslabón de la cadena, que es el más simple. Reemplazando v_j por su definición:

$$v_j(n) = \sum_{i=0}^N w_{ji}(n) y_i(n)$$

Al derivar esa suma respecto de **un** peso w_{ji} , todos los términos con índice distinto de i son **constantes** —no contienen la variable contra la que se deriva— y su derivada es cero. Sobrevive un solo término:

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = \frac{\partial [w_{ji}(n) y_i(n)]}{\partial w_{ji}(n)} = y_i(n)$$

Es el caso de derivar $3x$ respecto de x : queda la constante que multiplica. Acá la constante es $y_i(n)$, la entrada que llega por esa conexión.

OJO — y_i no es y_j , y se diferencian por un solo subíndice

Es la sutileza que se remarca en clase, y es fácil de pasar por alto porque las dos letras son iguales:

- $y_i(n)$ es **la entrada** a la neurona j por la conexión i —o sea, la salida de la neurona i de la **capa anterior**.
- $y_j(n)$ es **la salida** de la neurona j , la que sale de su función de activación.

Cuando aparezcan los superíndices de capa la distinción se vuelve obvia: $y_i^{(p-1)}$ contra $y_j^{(p)}$. Mientras tanto, la regla mental es: **el subíndice del peso te dice quién es quién**. En w_{ji} , el j es la neurona que recibe y el i es de dónde viene.

Claves de la sección 7

Clave	Qué tenés que poder responder
La derivada	Por qué de toda la sumatoria sobrevive un solo término
El resultado	Qué es y_i concretamente
y_i vs. y_j	Cuál es entrada y cuál es salida
Subíndices de w_{ji}	Cuál indica la neurona que recibe

8. El gradiente de error local instantáneo: δ_j

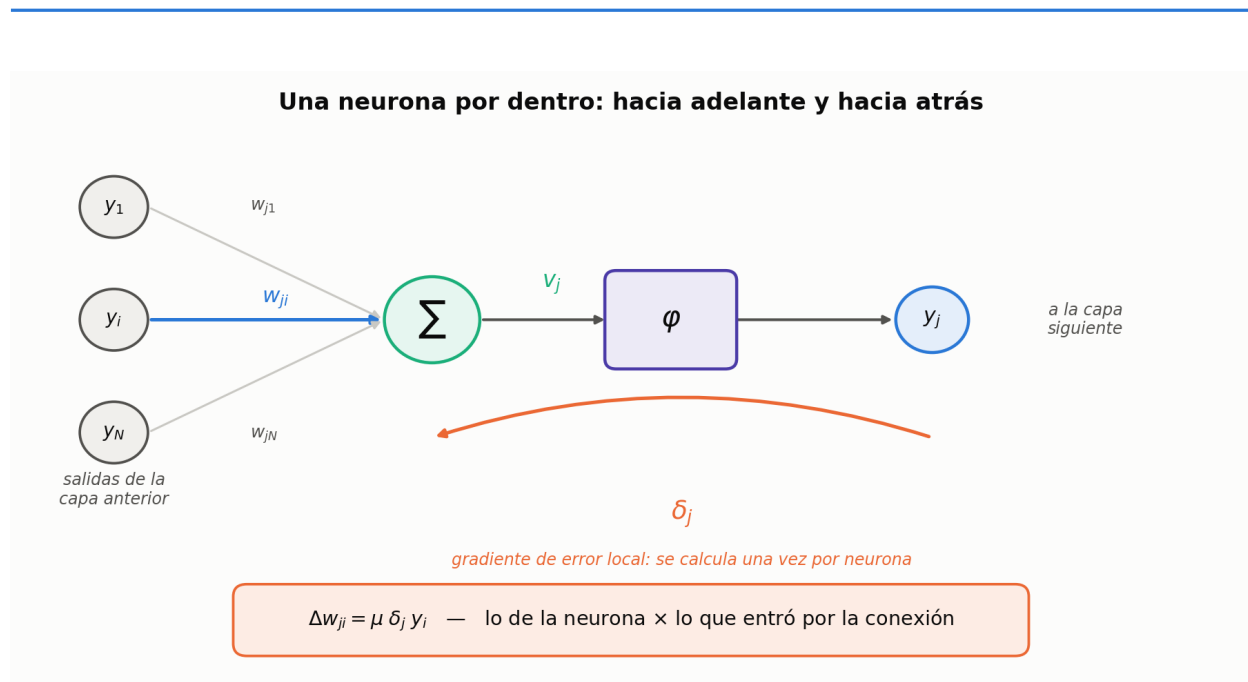


Figura 5: Todo lo que pasa en una neurona: hacia adelante la entrada se pesa, se suma y pasa por φ ; hacia atrás vuelve un único número, el δ_j .

Los tres factores restantes de la cadena se agrupan en una sola cantidad, que es **la pieza central de todo back-propagation**:

$$\delta_j(n) = -\frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}$$

Cada palabra del nombre está justificada:

- **gradiente**, porque es una derivada: es una parte del gradiente global que se viene calculando;
- **local**, porque es de **esa** neurona j en particular;
- **instantáneo**, porque está calculado en la iteración n .

En la práctica se lo llama simplemente **delta**.

IDEA DE FONDO — por qué conviene agrupar así y no de otra forma

El corte no es arbitrario. δ_j junta **todo lo que depende de la neurona** y deja afuera **lo que depende**

de la conexión. Por eso la actualización termina teniendo la forma

$$\Delta w_{ji} = \mu \cdot \underbrace{\delta_j}_{\text{de la neurona}} \cdot \underbrace{y_i}_{\text{de la conexión}}$$

y por eso el δ **se calcula una sola vez por neurona** y después se reutiliza para todos los pesos que entran a ella. Si agruparas de otro modo, tendrías que recalcular lo mismo una vez por conexión. Es también lo que hace posible propagar hacia atrás: lo único que una capa necesita de la capa siguiente son sus δ .

OJO — el signo menos del δ

En el desarrollo de clase el δ aparece primero **sin** el signo menos y enseguida **con** él. La versión que se usa de acá en adelante es **con el menos**, y no es un detalle: es lo que hace que la actualización quede **sumando** en vez de restando, absorbiendo el $-\mu$ de la regla del gradiente. Si copiás la versión sin el menos y después usás la fórmula final, la red aprende en la dirección contraria.

Notá además que se escribió $\frac{\partial \xi}{\partial y_j}$ de una sola vez en lugar de $\frac{\partial \xi}{\partial e_j} \frac{\partial e_j}{\partial y_j}$. Es la misma cosa: sólo se compactaron dos eslabones de la cadena para dejar a la vista lo que importa.

Claves de la sección 8

Clave	Qué tenés que poder responder
Definición	Escribir δ_j y decir qué factores agrupa
Las tres palabras	Justificar “gradiente”, “local” e “instantáneo”
Por qué ese corte	Qué es de la neurona y qué es de la conexión
El signo	Qué pasa si lo copiás sin el menos

9. La derivada de la sigmoide simétrica

De los dos factores del δ , uno se puede resolver ya: $\partial y_j / \partial v_j$ es simplemente **la derivada de la función de activación**.

OJO — acá se hace una simplificación

Se asume que **todas las neuronas de la red tienen la misma función de activación**. Existen redes con funciones distintas por neurona, o con parámetros distintos; en ese caso esta derivada habría que calcularla neurona por neurona. Con la simplificación, se calcula **una vez** y sirve para toda la red.

Partiendo de $\varphi(v) = \frac{2}{1 + e^{-v}} - 1$ y derivando respecto de v :

$$\frac{\partial y_j}{\partial v_j} = \frac{2 e^{-v_j}}{(1 + e^{-v_j})^2} = 2 \underbrace{\frac{1}{1 + e^{-v_j}}}_{(1)} \underbrace{\frac{e^{-v_j}}{1 + e^{-v_j}}}_{(2)}$$

El paso astuto está en el factor (2): **se suma y se resta 1** en el numerador —que es lo mismo que no hacer nada— para poder partirlo en dos:

$$\frac{e^{-v_j}}{1 + e^{-v_j}} = \frac{-1 + 1 + e^{-v_j}}{1 + e^{-v_j}} = 1 - \frac{1}{1 + e^{-v_j}}$$

Y ahora el reemplazo que resuelve todo. De la definición de la sigmoide,

$$y_j = \frac{2}{1 + e^{-v_j}} - 1 \quad \Leftrightarrow \quad \frac{1}{1 + e^{-v_j}} = \frac{y_j + 1}{2}$$

Metiendo eso en los dos factores, **la exponencial desaparece**:

$$\frac{\partial y_j}{\partial v_j} = 2 \cdot \frac{y_j + 1}{2} \left(1 - \frac{y_j + 1}{2} \right) = (y_j + 1) \frac{2 - y_j - 1}{2}$$

$$\boxed{\frac{\partial y_j}{\partial v_j} = \varphi'(v_j) = \frac{1}{2} (1 + y_j)(1 - y_j)}$$

IDEA DE FONDO — la derivada se escribe con la salida, no con la entrada

Éste es el motivo real por el que se elige una sigmoide y no otra función suave cualquiera. La derivada **no depende de v_j** : depende de y_j , que es un número que la propagación hacia adelante **ya calculó**. No hay que volver a evaluar ninguna exponencial ni guardar los v : con las salidas de la pasada hacia adelante alcanza. Es fácil de recordar además: *un medio, por uno más la salida, por uno menos la salida*.

OJO — el orden de los factores define el signo, y esto sí se presta a error

Es $\frac{1}{2}(1+y)(1-y)$, **no** $\frac{1}{2}(1+y)(y-1)$. Las dos expresiones son idénticas salvo el signo, y la segunda da siempre **negativa** —lo cual es imposible, porque la sigmoide es creciente y su derivada tiene que ser positiva en todo punto. Comprobado contra la derivada numérica:

v	y	num.	1/2(1+y)(1-y)	1/2(1+y)(y-1)
-3.00	-0.9051	0.09035	0.09035	-0.09035
-1.00	-0.4621	0.39322	0.39322	-0.39322
0.00	0.0000	0.50000	0.50000	-0.50000
1.00	0.4621	0.39322	0.39322	-0.39322
3.00	0.9051	0.09035	0.09035	-0.09035

Maximo error de 1/2(1+y)(1-y) frente a la derivada numerica: 2.660e-10

El control mental de tres segundos: **en $v = 0$ la derivada tiene que dar +0,5**. Si tu fórmula da -0,5, tenés los factores dados vuelta.

OJO — dónde se fue el parámetro b

Las diapositivas hacen la deducción con $b = 1$ y por eso b no aparece en el resultado. En el caso general, $\varphi(v) = \frac{2}{1+e^{-bv}} - 1$ y la derivada es

$$\varphi'(v_j) = \frac{b}{2} (1 + y_j)(1 - y_j)$$

Comprobado: para $v = 0,7$ y $b = 2$, la derivada numérica da 0,634740, que es exactamente $\frac{b}{2}(1 + y)(1 - y)$ y el **doble** de lo que daría la fórmula sin el b . Con $b = 1$ las dos coinciden, y por eso en esta materia no molesta — pero si algún día cambiás b , el factor tiene que estar.

Qué significa esta derivada

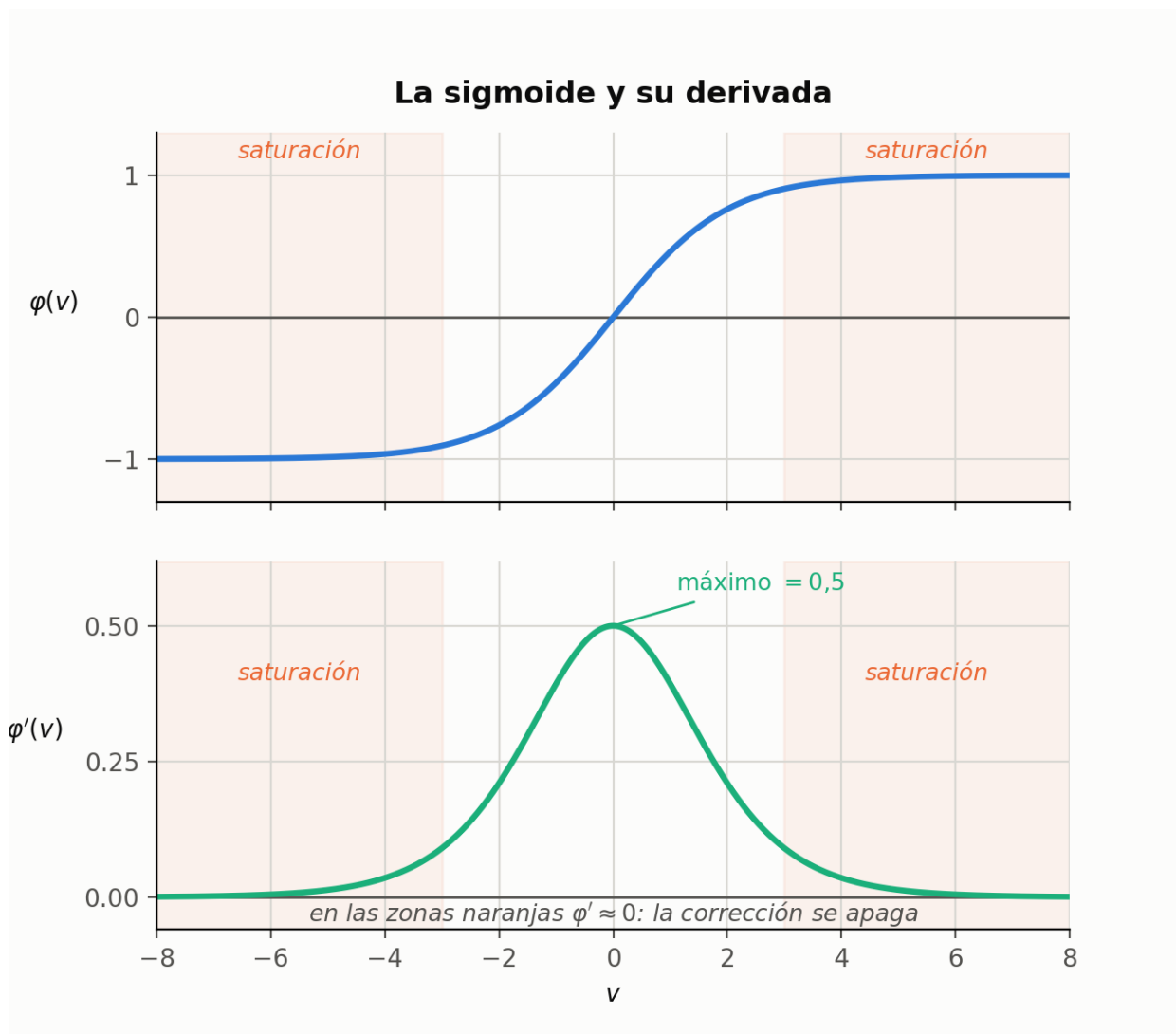


Figura 6: Arriba la sigmoide, abajo su derivada. La corrección de los pesos es proporcional a la curva de abajo.

La derivada vale como máximo 0,5, justo en $v_j = 0$ —donde la neurona está indecisa— y se desploma hacia cero en los extremos.

Como φ' multiplica al δ , y el δ multiplica a la corrección, esto significa que **una neurona saturada casi no corrige sus pesos**, por más grande que sea su error.

Es el mismo fenómeno que anticipaba el recuadro de la sección 4 sobre inicializar los pesos chicos, ahora con la cuenta que lo explica: si $\|w\|$ es grande, v es grande, $\varphi' \approx 0$ y el aprendizaje se apaga.

Y explica algo más, que aparece cuando se apilan capas: como cada capa hacia atrás multiplica por otro $\varphi' \leq 0,5$, las correcciones se van achicando a medida que se alejan de la salida. Las primeras capas aprenden más lento que las últimas — un problema real de las redes profundas, y acá se ve

de dónde sale.

Claves de la sección 9

Clave	Qué tenés que poder responder
Simplificación	Qué se asume sobre las funciones de activación
El truco	Para qué se suma y se resta 1
El reemplazo	Cómo se pasa de $\frac{1}{1+e^{-v}}$ a $\frac{y+1}{2}$
Resultado	Escribir φ' y el control en $v = 0$
Por qué importa	Qué ventaja tiene que dependa de y y no de v
Saturación	Por qué una neurona saturada no aprende

10. La regla de actualización

Juntando todo lo que se resolvió:

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = -\delta_j(n) y_i(n) \quad \Rightarrow \quad \Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

$$\boxed{\Delta w_{ji}(n) = \mu \delta_j(n) y_i(n)} \quad \text{con} \quad \delta_j(n) = -\frac{\partial \xi(n)}{\partial y_j(n)} \cdot \frac{1}{2}(1 + y_j(n))(1 - y_j(n))$$

El signo menos de la regla del gradiente **se canceló** con el del δ : por eso la corrección queda sumando.

PARA LA DEFENSA — es la misma forma que el LMS del perceptrón simple

Comparalas:

	corrección
LMS (perceptrón simple)	$\Delta w = 2\mu e(n) x(n)$
Back-propagation	$\Delta w_{ji} = \mu \delta_j(n) y_i(n)$

Es la misma estructura: velocidad de aprendizaje $\times$ algo local de la neurona $\times$ lo que entró por esa conexión. Lo único que cambió es que el error crudo e fue reemplazado por el δ , y la entrada x por la salida de la capa anterior y_i . Dicho de otro modo: **back-propagation no es un algoritmo nuevo, es el LMS con un error corregido**. El δ es “cuánto error le toca a esta neurona”, y calcularlo es todo el problema — trivial en la capa de salida, y el asunto central en las ocultas.

De la fórmula queda **un solo factor sin resolver**: $-\frac{\partial \xi(n)}{\partial y_j(n)}$. Y ahí está el quiebre de la unidad, porque la respuesta **depende de en qué capa esté la neurona**.

Claves de la sección 10

Clave	Qué tenés que poder responder
Fórmula final	Escribir $\Delta w_{ji} = \mu \delta_j y_i$
El signo	Por qué la corrección quedó sumando
Paralelo con LMS	Qué papel juega el δ y qué papel la y_i
Lo que falta	Cuál es el único factor sin resolver y de qué depende

11. Back-propagation en la capa de salida

Éste es el caso fácil, y por eso se hace primero. Con los superíndices de capa puestos, lo que se busca es el ajuste de los pesos de la capa III:

$$\Delta w_{ji}^{III}(n) = \mu \delta_j^{III}(n) y_i^{II}(n)$$

OJO — quién es y_i^{II} acá

Es la **entrada** a la capa III, o sea **la salida de la capa II**. Lo que sale de las neuronas de la capa II es lo que entra a las de la capa III. Es la misma sutileza y_i / y_j de la sección 7, ahora con el superíndice que la vuelve evidente: la entrada lleva el superíndice de la capa **anterior**.

Falta entonces δ_j^{III} , que por definición es

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial y_j^{III}(n)} \cdot \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

La segunda mitad ya está resuelta —es la derivada de la sigmoide de la sección 9—, así que sólo queda $\partial \xi / \partial y_j^{III}$. Y acá se vuelve a abrir el paso que en la sección 8 se había compactado:

$$\frac{\partial \xi(n)}{\partial y_j^{III}(n)} = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j^{III}(n)}$$

Primer factor. Reemplazando $\xi(n) = \frac{1}{2} \sum_k e_k^2(n)$ y derivando respecto de e_j : de toda la sumatoria, **los términos con $k \neq j$ son constantes** y su derivada es cero. Sobrevive sólo el término j , donde baja el 2 del cuadrado —**y ese 2 se simplifica con el $\frac{1}{2}$** :

$$\frac{\partial \xi(n)}{\partial e_j(n)} = e_j(n)$$

Segundo factor. Como $e_j(n) = d_j(n) - y_j^{III}(n)$, y d_j es una constante:

$$\frac{\partial e_j(n)}{\partial y_j^{III}(n)} = -1$$

Juntando. El -1 se cancela con el signo menos de la definición del δ :

$$\delta_j^{III}(n) = \frac{1}{2} e_j(n) (1 + y_j^{III}(n))(1 - y_j^{III}(n)) \quad \star$$

Y reemplazando en la actualización, absorbiendo el $\frac{1}{2}$ y el μ en una sola constante η :

$$\Delta w_{ji}^{III}(n) = \eta \underbrace{e_j(n)}_{\text{error}} \underbrace{(1 + y_j^{III}(n))(1 - y_j^{III}(n))}_{\text{derivada de } \varphi} \underbrace{y_i^{II}(n)}_{\text{entrada}}$$

PARA LA DEFENSA — la estructura de la fórmula es la respuesta

Si te piden la actualización de la capa de salida y te olvidás de un factor, reconstruila por su estructura, que se dice en clase tal cual: **velocidad de aprendizaje** $\times$ **error** $\times$ **derivada de la función de activación** $\times$ **entrada**. Cuatro piezas, en ese orden. Y es la misma estructura del LMS de la unidad anterior con un factor extra: **la derivada de la activación**, que es exactamente lo que allá no aparecía porque se había supuesto el caso lineal.

IDEA DE FONDO — la estrellita $\star$ no es decoración

El resultado de δ_j^{III} queda marcado con una estrella en las diapositivas porque **se vuelve a usar**: cuando toque derivar las capas ocultas, la cuenta va a llegar a una expresión que contiene exactamente esto, y en vez de recalcularla se la reemplaza por δ_k^{III} . Ése es el truco que convierte una cuenta larguísima en una fórmula recursiva — y, en la práctica, el que permite programar back-propagation como un bucle sobre las capas en vez de una fórmula distinta para cada una.

OJO — otra vez la constante cambia de nombre

Se pasó de μ a η para absorber el $\frac{1}{2}$ que venía del δ . Es el mismo movimiento que en la unidad del perceptrón simple, donde $\eta = 4\mu$. **La constante de aprendizaje se redefine cada vez que aparece un factor numérico**, así que no te preocupes por su valor: lo que hay que reconocer es la forma de la ecuación. Si en un parcial te aparece un $\frac{1}{2}$ de más o de menos, mirá si el enunciado usa μ o η .

Claves de la sección 11

Clave	Qué tenés que poder responder
y_i^{II}	Por qué la entrada lleva el superíndice de la capa anterior
$\partial \xi / \partial e_j$	Por qué sobrevive un solo término y adónde va el 2
$\partial e_j / \partial y_j$	Cuánto vale y con qué se cancela
δ_j^{III}	Escribirlo completo
Estructura	Los cuatro factores de Δw_{ji}^{III} , en orden
La estrellita	Por qué este resultado se guarda para después

12. Back-propagation en las capas ocultas

Acá se complica, y conviene decir por qué antes de meterse en la cuenta.

El punto de partida es **la misma ecuación**, sólo que con otro superíndice:

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

La entrada a la capa II es y_i^I , la salida de la capa I — que todavía **no** es la entrada de la red: para eso habría que ir una capa más atrás. Y el δ ahora es el de la capa II, que es lo que hay que encontrar.

12.1 Por qué esto es más difícil

El error se mide en **la capa de salida**, pero ahora hay que derivar respecto de una neurona de **la capa oculta**. Como se dice en clase: “*cada vez estamos más lejos de la salida final de la red*”. Para llegar del error hasta la neurona j hay que **atravesar la capa de salida**, y ese trayecto es toda la dificultad.

Arrancando de la definición del δ , la parte de la derecha ya está resuelta —es la derivada de la sigmoide de la sección 9— y lo que falta es la de la izquierda:

$$\delta_j^{II}(n) = -\frac{\partial \xi(n)}{\partial y_j^{II}(n)} \cdot \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

12.2 Los tres índices: ordenar esto antes de empezar

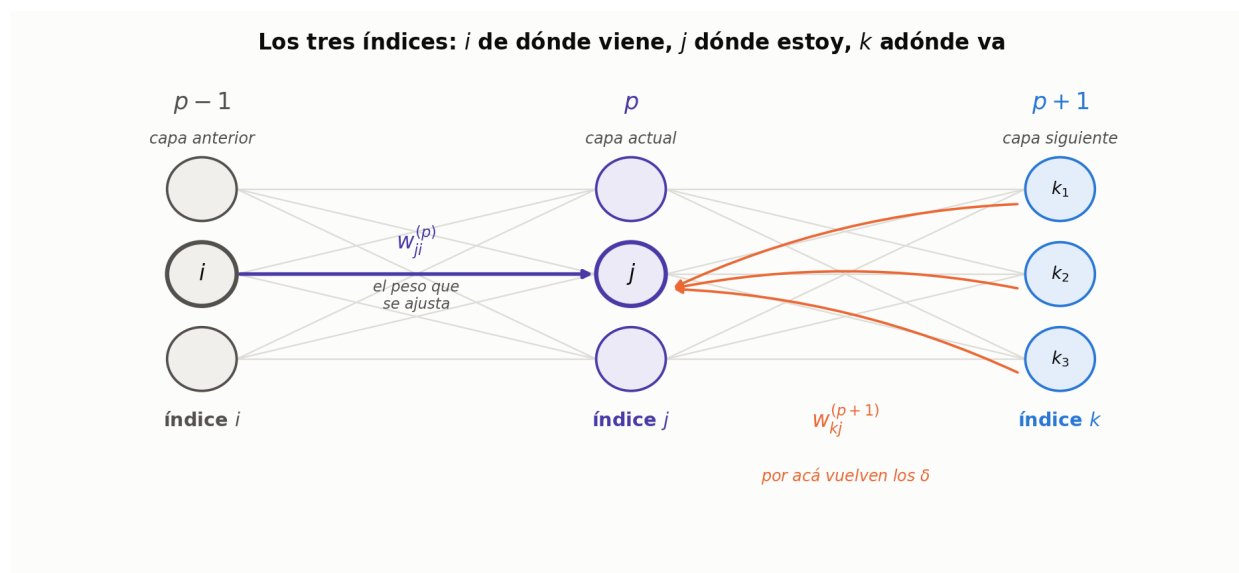


Figura 7: Cada índice vive en una capa distinta. Confundirlos es el error más común de toda la unidad.

Antes de la cuenta hay que fijar la notación, porque de acá en adelante conviven tres índices y en clase se insiste en no mezclarlos:

Índice	Recorre	Es
i	las neuronas de la capa anterior	de dónde viene la entrada
j	las neuronas de la capa actual (la oculta)	dónde estoy parado
k	las neuronas de la capa de salida	adónde va lo que produzco

OJO — la y que está adentro del error no es la y contra la que derivó

Es la advertencia que más se repite en esta clase. Dentro de $e_k = d_k - y_k^{III}$ **la salida que aparece es la de la capa III**, no la y_j^{II} de la capa oculta. Son dos cosas distintas que se escriben con la misma letra y sólo se distinguen por el superíndice y el subíndice. Justamente **porque no son la misma** es que hay que recorrer un camino de derivadas para conectarlas. Si fueran la misma, la cuenta se terminaría en un renglón.

12.3 Desarmar el criterio de error

Reemplazando $\xi(n) = \frac{1}{2} \sum_k e_k^2(n)$ y usando que **la derivada de una suma es la suma de las derivadas** —se saca el $\frac{1}{2}$ afuera y se entra con la derivada en la sumatoria—:

$$\delta_j^{II}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_k e_k^2(n) \right\}}{\partial y_j^{II}(n)} \frac{1}{2} (1 + y_j^{II})(1 - y_j^{II}) = -\frac{1}{2} \sum_k \frac{\partial e_k^2(n)}{\partial y_j^{II}(n)} \frac{1}{2} (1 + y_j^{II})(1 - y_j^{II})$$

Cada término de la suma es la derivada de algo elevado al cuadrado: baja el 2 —que se cancela con el $\frac{1}{2}$ — y queda la función por la derivada de la función:

$$\delta_j^{II}(n) = -\sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_j^{II}(n)} \frac{1}{2} (1 + y_j^{II})(1 - y_j^{II})$$

OJO — acá la sumatoria NO se colapsa

En la capa de salida, al derivar $\sum_k e_k^2$ respecto de e_j , sobrevivía **un solo término**. Acá no: se está derivando respecto de y_j^{II} , y la salida de esa neurona oculta **alimenta a todas las neuronas de la capa de salida**. Ninguno de los términos es constante, así que **la sumatoria queda**. Ésa es, en una frase, toda la diferencia entre el caso fácil y el difícil: en la salida el error de una neurona es asunto suyo; en una capa oculta, la neurona es corresponsable del error de todas las que alimenta.

12.4 El camino que hay que recorrer

Falta $\partial e_k / \partial y_j^{II}$, y no se puede derivar de una porque el error de la capa III no depende directamente de la salida de la capa II. Hay que atravesar la capa de salida con la regla de la cadena, eslabón por eslabón:

$$\frac{\partial e_k(n)}{\partial y_j^{II}(n)} = \underbrace{\frac{\partial e_k(n)}{\partial y_k^{III}(n)}}_{(1)} \underbrace{\frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)}}_{(2)} \underbrace{\frac{\partial v_k^{III}(n)}{\partial y_j^{II}(n)}}_{(3)}$$

que se lee: el error de la capa de salida respecto de la salida de la capa de salida; ésa respecto de su salida lineal; y esa salida lineal respecto de la salida de la capa oculta —que es justamente **la entrada** a la capa de salida—. Ahí ya se llegó a la capa que interesa.

Factor (1). Como $e_k(n) = d_k(n) - y_k^{III}(n)$ y d_k es constante:

$$\frac{\partial e_k(n)}{\partial y_k^{III}(n)} = -1$$

Factor (2). Es la derivada de la función de activación, ya conocida —pero **de la capa de salida**:

$$\frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)} = \frac{1}{2}(1 + y_k^{III}(n))(1 - y_k^{III}(n))$$

Factor (3). Como $v_k^{III}(n) = \sum_j w_{kj}^{III}(n) y_j^{II}(n)$, al derivar respecto de y_j^{II} pasa lo mismo que en la sección 7: todos los términos con índice distinto de j son constantes y se anulan, y del que sobrevive queda la constante que multiplica:

$$\frac{\partial v_k^{III}(n)}{\partial y_j^{II}(n)} = w_{kj}^{III}(n)$$

IDEA DE FONDO — se acumulan DOS derivadas de activación

Prestá atención a que ahora hay dos: la del factor (2), que es la de **la capa de salida** y va evaluada en y_k^{III} , y la que se viene arrastrando desde la definición del δ , que es la de **la capa oculta** y va evaluada en y_j^{II} . En clase se las llama “*la colita que venimos arrastrando*”. Y ésta es la razón de fondo por la que las capas profundas aprenden lento: **cada capa que se retrocede agrega otro factor** $\varphi' \leq 0,5$, así que la corrección se multiplica por un número menor que uno una vez por capa.

12.5 Juntando todo

Reemplazando los tres factores, el -1 del factor (1) se cancela con el signo menos de adelante:

$$\delta_j^{II}(n) = \sum_k e_k(n) \frac{1}{2}(1 + y_k^{III})(1 - y_k^{III}) w_{kj}^{III}(n) \cdot \frac{1}{2}(1 + y_j^{II})(1 - y_j^{II})$$

Ésta ya es la fórmula del gradiente de error local en la capa oculta. Pero mirala con atención antes de seguir.

12.6 El reemplazo que es el corazón del método

De la capa de salida (sección 11) teníamos, marcado con $\star$:

$$\delta_k^{III}(n) = \frac{1}{2} e_k(n) (1 + y_k^{III}(n))(1 - y_k^{III}(n))$$

Eso es exactamente lo que quedó adentro de la sumatoria. Reemplazando:

$$\delta_j^{II}(n) = \left[\sum_k \delta_k^{III}(n) w_{kj}^{III}(n) \right] \frac{1}{2} (1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

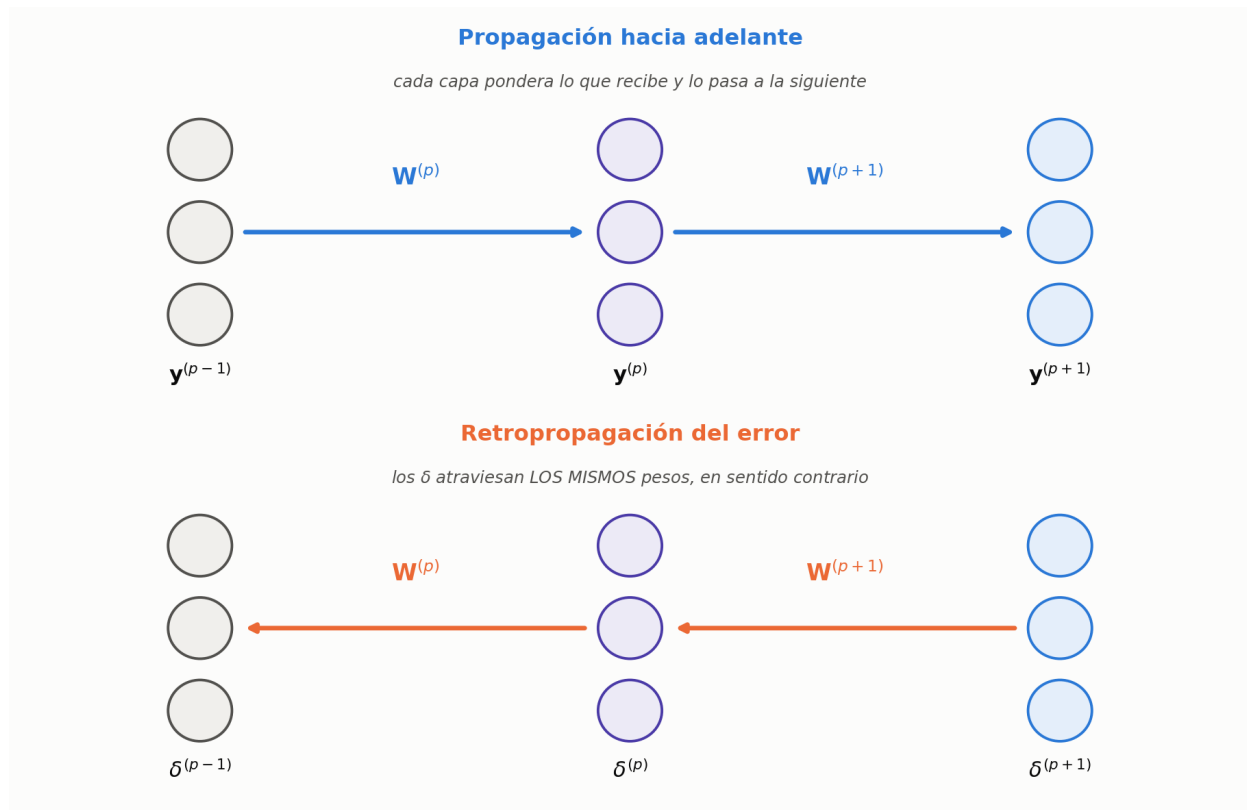


Figura 8: Los mismos pesos que llevan la información hacia adelante llevan el error hacia atrás.

PARA LA DEFENSA — qué significa ese corchete

Es el punto que en clase se marca como “una cuestión clave, el corazón del método”, y no es sólo eficiencia de cálculo: tiene contenido conceptual. Mirá qué es $\sum_k \delta_k^{III} w_{kj}^{III}$: un **promedio ponderado de los δ de la capa de salida, pesado por los pesos de la capa de salida**. Es decir, se están haciendo pasar los δ de la capa III a través de los pesos que unen la capa III con la capa II, pero en sentido contrario. Y eso es **exactamente el espejo de la propagación hacia adelante**: allá las entradas atravesaban los pesos para producir la salida de la capa siguiente; acá los δ atraviesan los mismos pesos, de atrás para adelante, para producir el δ de la capa anterior. **De ahí el nombre: retropropagación**. Un detalle que suma: el sentido se invierte pero los pesos son **los mismos**. Hacia adelante se usan las **filas** de W ; hacia atrás, las **columnas** — o sea W^T . No hay una segunda red para el camino de vuelta.

Y la actualización de los pesos de la capa oculta queda:

$$\Delta w_{ji}^{II}(n) = \eta \underbrace{\left[\sum_k \delta_k^{III}(n) w_{kj}^{III}(n) \right]}_{\text{error retropropagado}} \underbrace{(1 + y_j^{II}(n))(1 - y_j^{II}(n))}_{\text{derivada de } \varphi \text{ acá}} \underbrace{y_i^I(n)}_{\text{entrada}}$$

La misma estructura de siempre: velocidad de aprendizaje $\times$ error $\times$ derivada de la activación $\times$ entrada. Lo único que cambió es que el “error” ya no es e_j sino el error **retropropagado** desde la capa siguiente.

Claves de la sección 12

Clave	Qué tenés que poder responder
Los tres índices	Qué recorre i , qué recorre j , qué recorre k
Por qué es difícil	Dónde se mide el error y dónde hay que derivar
La sumatoria	Por qué no se colapsa como en la capa de salida
Los tres factores	Cuánto vale cada uno de (1), (2) y (3)
Dos φ'	Cuál va evaluada en y_k^{III} y cuál en y_j^{II}
El reemplazo	Cómo aparece δ_k^{III} adentro de la cuenta
El corchete	Qué significa conceptualmente, y por qué “retro”
$\mathbf{w}^T$	Por qué el camino de vuelta usa los mismos pesos

13. La generalización: una capa p cualquiera

La ecuación de la capa oculta no depende de que la red tenga tres capas. Renombrando: si p es **la capa en la que estoy parado**, entonces $p - 1$ es de donde viene la entrada y $p + 1$ es la capa siguiente:

$$\Delta w_{ji}^{(p)}(n) = \eta \langle \delta^{(p+1)}, \mathbf{w}_j^{(p+1)} \rangle (1 + y_j^{(p)}(n))(1 - y_j^{(p)}(n)) y_i^{(p-1)}(n)$$

Pieza por pieza:

Pieza	Qué es
η	velocidad de aprendizaje
$\langle \delta^{(p+1)}, \mathbf{w}_j^{(p+1)} \rangle$	el error retropropagado desde la capa siguiente
$(1 + y_j^{(p)}(n))(1 - y_j^{(p)}(n))$	derivada de la activación en la capa actual
$y_i^{(p-1)}$	la entrada a la capa p , o sea la salida de la capa $p - 1$

OJO — qué vector es $\mathbf{w}_j^{(p+1)}$

No es la fila de pesos de una neurona: es el vector de los pesos que **salen** de la neurona j hacia todas las neuronas de la capa $p+1$. En la matriz $\mathbf{W}^{(p+1)}$ eso es **la columna** j , no una fila. Por eso el producto interno se escribe con $\delta^{(p+1)}$, que tiene una componente por cada neurona de esa capa. Es la forma compacta de decir $\sum_k \delta_k^{(p+1)} w_{kj}^{(p+1)}$.

PARA LA DEFENSA — por qué esta ecuación cierra la unidad

Porque **no importa cuántas capas tenga la red**. Con esta única fórmula se pueden ajustar los pesos de todas las capas: se empieza por la de salida, donde el δ sale del error verdadero, y de ahí para atrás cada capa arma su δ con los de la capa siguiente. Es un **bucle**, no una fórmula por capa. Si te piden “explicá back-propagation”, esto es el final del camino: una regla recursiva que hace bajar el error por una red de profundidad arbitraria.

Claves de la sección 13

Clave	Qué tenés que poder responder
La fórmula general	Escribirla e identificar sus cuatro piezas
$p-1, p, p+1$	Qué papel cumple cada capa
$\mathbf{w}_j^{(p+1)}$	Por qué es una columna y no una fila
Por qué generaliza	Por qué sirve para cualquier cantidad de capas

14. El algoritmo completo, paso a paso

Todo lo deducido hasta acá son ecuaciones sueltas. Esta sección las pone en orden de ejecución: qué se calcula primero, qué después, y con qué valores.



Figura 9: Los cuatro pasos y el bucle que los repite. El paso 4 nunca se hace antes de terminar el 3.

La red de ejemplo es chica a propósito: **2 entradas, 3 neuronas en la capa I, 2 en la capa II y 1 en la capa de salida**. Alcanza para que aparezcan todos los casos.

Paso 1 — Inicialización

Todos los pesos, al azar y **con valores pequeños**: por ejemplo, uniformes en $[-0,5; +0,5]$.

Ya vimos el motivo en la sección 9, pero conviene tenerlo a mano: con pesos grandes, v arranca grande, la neurona arranca **saturada**, $\varphi' \approx 0$ y la red no aprende. Los valores chicos la dejan en la zona donde la derivada es máxima.

Paso 2 — Propagación hacia adelante

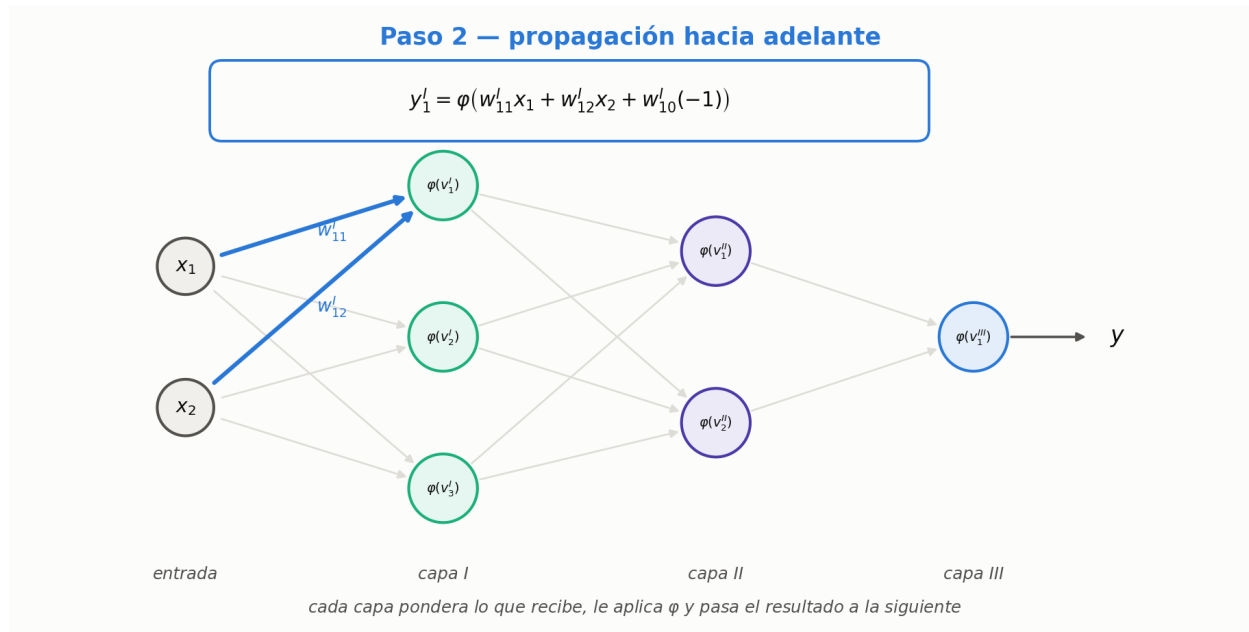


Figura 10: Se entra un patrón y se calcula la salida de cada neurona, capa por capa.

Entra un patrón (x_1, x_2) y se recorre la red de izquierda a derecha. Para la primera neurona de la capa I:

$$y_1^I = \varphi(w_{11}^I x_1 + w_{12}^I x_2 + w_{10}^I (-1))$$

donde el superíndice indica la **capa** y el subíndice, la **neurona** dentro de esa capa. Lo mismo para y_2^I e y_3^I . Después la capa II toma como entrada las tres salidas de la capa I, y la capa III toma las dos de la capa II:

$$y_1^{II} = \varphi(w_{11}^{II} y_1^I + w_{12}^{II} y_2^I + w_{13}^{II} y_3^I + w_{10}^{II} (-1)) \quad y = y_1^{III} = \varphi(w_{11}^{III} y_1^{II} + w_{12}^{III} y_2^{II} + w_{10}^{III} (-1))$$

OJO — el sesgo está aunque no se dibuje

En el esquema no aparece, para no ensuciarlo. Pero **todas** las neuronas de **todas** las capas tienen su peso de sesgo multiplicando a -1 : $w_{10}^I, w_{20}^I, w_{30}^I, w_{10}^{II}, w_{20}^{II}, w_{10}^{III}$. Son seis pesos más que hay que inicializar, usar y actualizar.

IDEA DE FONDO — hasta acá no se entrenó nada

El paso 2 es puro cálculo: se propagó una entrada y salió un número. **Ningún peso se movió.** Lo que sí queda es un subproducto imprescindible: las salidas y_j^I, y_j^{II}, y^{III} de todas las neuronas, que hay que **guardar** porque los pasos 3 y 4 las van a usar.

Paso 3 — Propagación hacia atrás

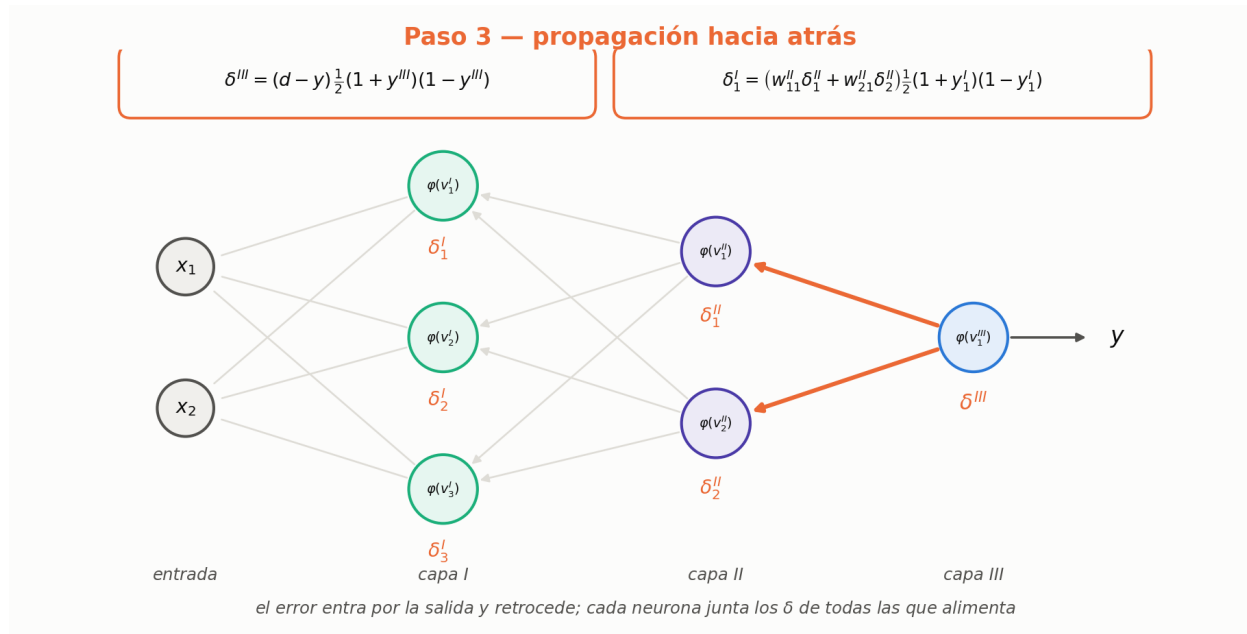


Figura 11: El error entra por la salida y retrocede convertido en δ .

Ahora se recorre la red de derecha a izquierda calculando un δ por neurona.

Capa de salida (una sola neurona, así que un solo δ):

$$\delta^{III} = \underbrace{(d - y)}_{\text{el error}} \cdot \frac{1}{2} (1 + y^{III})(1 - y^{III})$$

Capa II. Se propaga δ^{III} hacia atrás multiplicándolo por el peso que une cada neurona con la de salida:

$$\delta_1^{II} = w_{11}^{III} \delta^{III} \cdot \frac{1}{2} (1 + y_1^{II})(1 - y_1^{II}) \quad \delta_2^{II} = w_{12}^{III} \delta^{III} \cdot \frac{1}{2} (1 + y_2^{II})(1 - y_2^{II})$$

Capa I. Acá la suma tiene **dos** términos, porque la neurona 1 de la capa I alimenta a **las dos** neuronas de la capa II:

$$\delta_1^I = \left(w_{11}^{II} \delta_1^{II} + w_{21}^{II} \delta_2^{II} \right) \cdot \frac{1}{2} (1 + y_1^I)(1 - y_1^I)$$

IDEA DE FONDO — cuántos términos tiene la suma, sin pensar

Es la regla práctica que hace todo esto mecánico: **la sumatoria del δ de una neurona tiene tantos**

términos como neuronas haya en la capa siguiente. En esta red: la capa II mira a una sola neurona de salida → **un término** (por eso arriba no hay sumatoria, es un producto). La capa I mira a dos neuronas de la capa II → **dos términos**. Y el otro chequeo: la derivada de activación de afuera va siempre evaluada en **la salida de la neurona cuyo δ estás calculando**, nunca en la de la capa siguiente.

Al terminar el paso 3 hay un δ por neurona: $\delta_1^I, \delta_2^I, \delta_3^I, \delta_1^{II}, \delta_2^{II}, \delta^{III}$. Seis números, uno por neurona. **Todavía ningún peso se movió.**

Paso 4 — Adaptación de los pesos

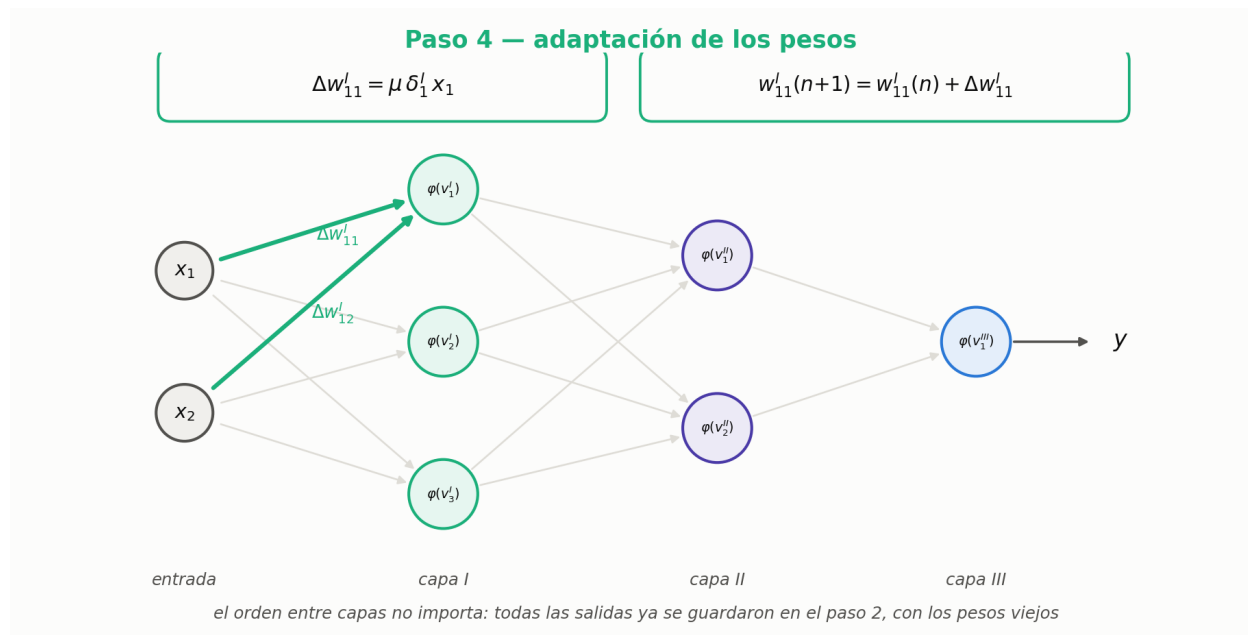


Figura 12: Recién acá cambian los pesos, y todos con los mismos valores guardados del paso 2.

Ahora sí. Para cada peso:

$$\Delta w_{11}^I = \mu \delta_1^I x_1 \qquad w_{11}^I(n+1) = w_{11}^I(n) + \Delta w_{11}^I$$

OJO — Δw no es el peso nuevo, es el cambio

Es la confusión más común y en clase se la repite varias veces: lo que sale de la fórmula es **el ajuste**, y hay que **sumárselo al peso actual**. El peso nuevo, el que va a actuar la próxima vez que entre un patrón, es $w(n) + \Delta w$.

Los pesos de sesgo se actualizan igual que los demás, usando el δ de su propia neurona y recordando que su entrada vale -1 :

$$\Delta w_{10}^I = \mu \delta_1^I (-1)$$

PARA LA DEFENSA — por qué el orden de actualización no importa

Éste es el punto más fino de la clase y se explica medio de corrido, así que vale desarmarlo. La

pregunta natural es: si actualizo primero la capa I, ¿los pesos nuevos no cambian las salidas que después necesito para actualizar la capa II? **No**, y la razón es que las salidas $y_j^{(p)}$ **ya están calculadas y guardadas** desde el paso 2, con los pesos viejos. No se recalculan. Lo mismo con los δ : todos salieron del paso 3, también con los pesos viejos. Por eso el paso 4 puede recorrer las capas en cualquier orden —de adelante hacia atrás, de atrás hacia adelante, saltado— y da **exactamente el mismo resultado**. **La precisión que falta:** eso vale porque los pasos 3 y 4 están **separados**. Si en cambio fueras intercalando —actualizar la capa III y después usar esos pesos nuevos para calcular δ^{II} — el resultado sí cambiaría, y estaría mal: el gradiente está definido en el punto $w(n)$, no en un punto a medio mover. Primero todos los δ , después todos los Δw . Ése es el orden que sí importa.

Paso 5 — Siguiente patrón, y épocas

Todo lo anterior fue **para un solo patrón**. Se toma el siguiente del archivo de entrenamiento y se repite: adelante, atrás, ajuste. Y así con todos.

IDEA DE FONDO — qué es una época

Una **época** es una pasada completa por todos los patrones de entrenamiento, cada uno una vez. Un entrenamiento típico son 10, 20, 50 o 100 épocas: al conjunto se le muestran los datos muchas veces, y en cada pasada los pesos se corrigen un poco más. Notá que los pesos se actualizan **después de cada patrón**, no al final de la época. Eso lo hace un entrenamiento **estocástico** (o *por patrón*), consistente con el “error instantáneo” $\xi(n)$ de la sección 5: nunca se promedió sobre el conjunto, y por eso conviene μ chico, para que el ruido de patrón a patrón se promedie solo.

Claves de la sección 14

Clave	Qué tenés que poder responder
Los cinco pasos	Enumerarlos en orden
Inicialización	Por qué los valores chicos
Qué guarda el paso 2	Qué se necesita después y por qué no se recalcula
Cantidad de términos Δw vs. w	Cuántos suma el δ de una neurona y por qué Qué devuelve la fórmula y qué hay que hacer con eso
Orden de actualización	Por qué da igual, y qué orden sí importa
Época	Qué es y cuántas se usan
Estocástico	Cuándo se actualizan los pesos y qué implica para μ

15. Cierre: el recorrido completo de la unidad

Vale la pena mirar el camino entero de una vez, porque cada pieza contesta la pregunta que dejó abierta la anterior:

La pregunta	La respuesta
¿Qué puede resolver un perceptrón simple?	Sólo problemas linealmente separables
¿Y si no lo son?	Se combinan varios: la capa oculta cambia la representación
¿Cuántas capas hacen falta?	Una: semiplanos. Dos: regiones convexas. Tres: cualquier región
¿Cómo se encuentran los pesos?	Descenso por gradiente sobre el error cuadrático
¿Por qué no sirve sgn?	No es derivable: se cambia por la sigmoide simétrica
¿Cuánto corrige cada peso?	$\Delta w_{ji} = \mu \delta_j y_i$
¿Y el δ de una neurona oculta, si no tiene error propio?	Se arma con los δ de la capa siguiente: retropropagación
¿En qué orden se hace todo?	Adelante, atrás, ajustar; patrón a patrón; muchas épocas

PARA LA DEFENSA — la unidad en una frase

Back-propagation es el método del gradiente del perceptrón simple, aplicado a una red de varias capas, con dos agregados: la función de activación pasa a ser derivable, y el error de las neuronas ocultas —que nadie mide— se reconstruye haciendo pasar los δ de la capa siguiente por los mismos pesos, en sentido inverso.

Formulario

Expresión	Qué es
$\mathbf{v}^{(p)} = \mathbf{W}^{(p)} \mathbf{y}^{(p-1)}$	salida lineal de la capa p
$y_j^{(p)} = \varphi(v_j^{(p)})$	salida de la neurona j de la capa p
$\varphi(v) = \frac{2}{1 + e^{-bv}} - 1$	sigmoide simétrica (-1 a $+1$)
$\mathbf{W}^{(p)}$ es $M_p \times (M_{p-1} + 1)$	dimensiones, con la columna del sesgo
$e_j(n) = d_j(n) - y_j(n)$	error de la neurona de salida j
$\xi(n) = \frac{1}{2} \sum_{j=1}^M e_j^2(n)$	error cuadrático instantáneo de la red
$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$	regla del gradiente
$\frac{\partial \xi}{\partial w_{ji}} = \frac{\partial \xi}{\partial e_j} \frac{\partial e_j}{\partial y_j} \frac{\partial y_j}{\partial v_j} \frac{\partial v_j}{\partial w_{ji}}$	la cadena a resolver
$\frac{\partial v_j}{\partial w_{ji}} = y_i(n)$	la entrada que llega por esa conexión
$\varphi'(v_j) = \frac{1}{2}(1 + y_j)(1 - y_j)$	derivada de la sigmoide simétrica ($b = 1$)
$\varphi'(v_j) = \frac{b}{2}(1 + y_j)(1 - y_j)$	la misma, con b general
$\delta_j(n) = -\frac{\partial \xi}{\partial y_j} \varphi'(v_j)$	gradiente de error local instantáneo

Expresión	Qué es
$\Delta w_{ji}(n) = \mu \delta_j(n) y_i(n)$	la regla de actualización
$\delta_j^{III}(n) = \frac{1}{2} e_j (1 + y_j^{III})(1 - y_j^{III})$	delta de la capa de salida ★
$\Delta w_{ji}^{III} = \eta e_j (1 + y_j^{III})(1 - y_j^{III}) y_i^{II}$	ajuste en la capa de salida
$\delta_j^{II} = [\sum_k \delta_k^{III} w_{kj}^{III}] \frac{1}{2} (1 + y_j^{II})(1 - y_j^{II})$	delta de una capa oculta
$\Delta w_{ji}^{II} = \eta [\sum_k \delta_k^{III} w_{kj}^{III}] (1 + y_j^{II})(1 - y_j^{II}) y_i^I$	ajuste en la capa oculta
$\Delta w_{ji}^{(p)} =$ $\eta \langle \delta^{(p+1)}, \mathbf{w}_j^{(p+1)} \rangle (1 + y_j^{(p)})(1 - y_j^{(p)}) y_i^{(p-1)}$	la fórmula general
$w_{ji}(n+1) = w_{ji}(n) + \Delta w_{ji}(n)$	cómo se aplica el ajuste
$\Delta w_{j0}^{(p)} = \mu \delta_j^{(p)} (-1)$	ajuste del peso de sesgo

Errores típicos

- Contar las entradas como una capa. En esta materia las capas son **de neuronas**.
- Decir que tres capas “resuelven cualquier problema” sin la aclaración de que eso es **existencia**, no aprendizaje.
- Creer que con dos capas se puede hacer cualquier región. Sólo **convexas**.
- Olvidar el sesgo en las capas ocultas porque no está dibujado.
- Confundir $\mathbf{W}^{II}$ con “los pesos de la segunda capa vista desde la entrada”. Es la capa **de llegada**.
- Mezclar la sigmoide simétrica con la logística 0–1 al copiar fórmulas de otro libro.
- Perder de vista la distinción entre v (salida lineal) e y (salida). Toda la cadena se apoya ahí.
- Escribir el error total sin el cuadrado, o sin el $\frac{1}{2}$, y después no entender por qué no cierran las constantes.
- Confundir y_i con y_j en $\Delta w_{ji} = \mu \delta_j y_i$. La primera es la **entrada** por esa conexión; la segunda, la **salida** de la neurona.
- Escribir la derivada de la sigmoide como $\frac{1}{2}(1+y)(y-1)$. Da negativa, y una función creciente no puede tener derivada negativa.
- Usar el δ sin el signo menos y después aplicar $\Delta w = \mu \delta y_i$: la red corrige para el lado equivocado.
- Olvidar el factor b en φ' si se trabaja con $b \neq 1$.
- Recalcular la exponencial para obtener φ' . Ya la tenés: φ' se escribe con y_j , que la propagación hacia adelante calculó.
- Poner y_i^{III} donde va y_i^{II} en el ajuste de la capa de salida. La **entrada** lleva el superíndice de la capa anterior.
- Buscarle un e_j a una neurona oculta. No existe: no hay d_j para una neurona intermedia.
- Confundir μ con η . Difieren en las constantes absorbidas; lo que importa es la forma de la ecuación.
- Colapsar la sumatoria en la capa oculta como se hace en la de salida. Ahí **ninguno** de los términos es constante.
- Poner una sola derivada de activación en el δ de una capa oculta. Van **dos**: la de la capa siguiente (en y_k^{III}) y la de la capa actual (en y_j^{II}).
- Evaluar la derivada de la activación en la neurona equivocada. La del corchete va en k ; la de

afuera, en j .

- Tomar $\mathbf{w}_j^{(p+1)}$ como una fila de la matriz. Es **la columna j** : los pesos que salen de la neurona j .
- Tratar a Δw como si fuera el peso nuevo. Es **el cambio**: hay que sumárselo al peso actual.
- Recalcular las salidas de las neuronas con los pesos ya actualizados. Se usan las **guardadas del paso 2**.
- Intercalar el cálculo de los δ con la actualización de los pesos. Primero **todos** los δ , después **todos** los Δw .
- Olvidar de actualizar los pesos de sesgo, que no están dibujados pero existen en todas las neuronas.
- Confundir iteración con época: una época son **todos** los patrones, una vez cada uno.

Autoevaluación

1. Dibujá la tabla de regiones de decisión de memoria: tres filas, qué región genera cada una.
2. ¿Por qué dos capas no pueden separar las dos medialunas, por más neuronas que se pongan en la primera capa?
3. Según la convención de la cátedra, ¿de cuántas capas es la red del XOR? ¿Y contando como lo hace la mayoría de la bibliografía moderna?
4. Una capa de 7 neuronas recibe 12 entradas. ¿Qué dimensiones tiene su matriz de pesos y cuántos parámetros son?
5. Escribí las tres ecuaciones de la propagación hacia adelante para una red de tres capas.
6. ¿Qué problema concreto tiene sgn que la sigmoide no tiene?
7. ¿Qué pasa con el aprendizaje si los pesos se inicializan grandes? Explicalo con la forma de la sigmoide.
8. Justificá el cuadrado y el $\frac{1}{2}$ en $\xi(n)$, cada uno con su motivo.
9. Escribí la ecuación de la regla de la cadena y explicá qué representa cada uno de los cuatro factores.
10. Derivá $\partial v_j / \partial w_{ji}$ y explicá por qué sobrevive un solo término de la sumatoria.
11. ¿Qué agrupa el δ_j y por qué conviene cortar la cadena justo ahí?
12. Deducí $\varphi'(v) = \frac{1}{2}(1+y)(1-y)$ paso a paso, incluido el truco de sumar y restar 1.
13. ¿Cuánto vale φ' en $v = 0$? Usalo como control de signo.
14. Compará $\Delta w_{ji} = \mu \delta_j y_i$ con la regla del LMS: ¿qué reemplaza a qué?
15. ¿Por qué una neurona saturada casi no ajusta sus pesos?
16. Deducí δ_j^{III} paso a paso: los dos factores de $\partial \xi / \partial y_j$ y dónde se cancela cada signo.
17. Enunciá la estructura de Δw_{ji}^{III} en cuatro palabras, sin escribir la fórmula.
18. ¿Para qué queda marcado con $\star$ el resultado de δ_j^{III} ?
19. ¿Por qué el problema de las capas ocultas es más difícil que el de la capa de salida?
20. Explicá con la figura por qué el δ de una neurona oculta necesita una sumatoria.
21. Deducí δ_j^{II} completo, desde $\xi(n)$ hasta el corchete, sin saltar pasos.
22. ¿Por qué en la capa oculta la sumatoria no se colapsa y en la de salida sí?
23. Escribí los tres factores de $\partial e_k / \partial y_j^{II}$ y cuánto vale cada uno.
24. ¿Dónde aparecen las dos derivadas de activación y en qué neurona va evaluada cada una?
25. Explicá en qué sentido el corchete $\sum_k \delta_k^{III} w_{kj}^{III}$ es el espejo de la propagación hacia adelante.

26. Escribí la fórmula general para la capa p y explicá por qué sirve para cualquier profundidad.
27. Enumerá los cinco pasos del algoritmo de back-propagation.
28. En la red de ejemplo (2-3-2-1), ¿cuántos δ se calculan y cuántos pesos se actualizan por patrón?
29. ¿Cuántos términos tiene la sumatoria del δ de una neurona de la capa I? ¿Y de la capa II?
¿Por qué?
30. ¿Por qué el orden en que se actualizan las capas no cambia el resultado?
31. ¿Qué orden sí importa, y qué pasaría si lo invirtieras?
32. Diferencia entre iteración y época. ¿Cuándo se actualizan los pesos?